

Структура научной презентации

Простейший вариант

Mehmet Efe Kantoz

30.08.2026

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	История создания	7
3.2	Общая информация	7
3.3	Общая информация	8
3.4	Свойства	8
3.5	Методы анализа сетей Петри	8
3.6	Обедающие философы	9
3.7	Классическая постановка	9
3.8	Классическая постановка	10
3.9	Проблема взаимной блокировки (deadlock)	10
3.10	Решение проблемы	10
3.11	Связь с сетями Петри	10
3.12	Общие замечания	11
4	Выполнение лабораторной работы	12
4.1	Базовый эксперимент	12
4.2	Базовый эксперимент	13
4.3	Базовый эксперимент	13
4.4	Анимация процесса	15
4.5	Анимация процесса	16
4.6	Анимация процесса	16
4.7	Итоговый отчет	16
4.8	Итоговый отчет	17
4.9	Итоговый отчет	17
5	Выводы	19
	Список литературы	20

Список иллюстраций

4.1	Код модели	12
4.2	Базовая модель	13
4.3	Запуск	13
4.4	Код модели	14
4.5	notebook	14
4.6	График классической модели	14
4.7	График модели с арбитром	15
4.8	Код анимации	15
4.9	производные форматы	16
4.10	Код модели	16
4.11	Запуск	17
4.12	производные форматы	17
4.13	notebook	17
4.14	График	18

Список таблиц

1 Цель работы

- Построить сеть Петри для пяти философов, моделируя захват и освобождение вилок.
- Обнаружить состояние взаимной блокировки (deadlock), когда каждый философ взял одну вилку и ждёт вторую.
- Провести имитационное моделирование (стохастическое и детерминированное) и выявить наличие deadlock.
- Модифицировать сеть, чтобы предотвратить deadlock.
- Проанализировать результаты и оформить отчёт с графиками и анимацией.

2 Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook с параметрами.
- Интегрировать документацию с параметрами в формате Quarto в отчёт.

3 Теоретическое введение

3.1 История создания

Первую идею сетей Петри Карл Адам Петри сформулировал в возрасте 13 лет для описания химических реакций. Официальной датой рождения теории считается 1962 год, когда он защитил докторскую диссертацию «Kommunikation mit Automaten» («Взаимодействие с автоматами») в Боннском университете. За свою работу он получил премию Тьюринга (ACM Turing Award) — высшую награду в области информатики

3.2 Общая информация

Сеть Петри есть математический аппарат для моделирования дискретных систем. Графически она представляется как двудольный ориентированный граф двух типов вершин: позиции (круги) и переходы (прямоугольники). - Позиции (Places) суть пассивные элементы, описывающие состояние системы (например, наличие ресурса или выполнение условия). - Внутри позиции могут находиться фишки (tokens) — неотрицательное целое число, указывающее на количество ресурсов. - Переходы (Transitions) суть активные элементы, описывающие события или действия системы. - Они могут срабатывать, изменяя состояние модели. - Дуги (Arcs) суть направленные соединения между позициями и переходами (но не между двумя позициями или двумя переходами), которые показывают, как состояние влияет на события и наоборот. - Маркировка (Marking) есть распределение фишек

по позициям в определённый момент времени, то есть текущее состояние модели.

- Смена маркировок происходит при срабатывании переходов в соответствии с определёнными правилами.

3.3 Общая информация

Теория сетей Петри, появившаяся как инструмент для анализа химических процессов, сегодня является мощным и наглядным математическим аппаратом. Она незаменима везде, где нужно описать параллельные, асинхронные и распределённые системы. В её основе лежат всего четыре элемента, а богатство поведения возникает из их комбинации.

3.4 Свойства

Сети Петри используются не просто для моделирования, но и для проверки корректности системы. Для этого исследуются её важнейшие свойства:

- Ограниченность (Boundedness). Количество фишек в любой позиции сети никогда не превысит некоторого заданного предела K . Если $K = 1$, сеть называется безопасной.
- Активность (Liveness). Для любого перехода всегда существует достижимая маркировка, в которой он может сработать. Это гарантирует, что в системе не будет вечных блокировок.
- Достижимость (Reachability). Можно ли из начальной маркировки M_0 попасть в некоторую желаемую маркировку M_k .
- Сохраняемость (Conservation). Взвешенная сумма фишек по всем позициям остаётся постоянной (аналог закона сохранения массы или энергии).

3.5 Методы анализа сетей Петри

Для изучения этих свойств разработаны специальные методы: - Граф (дерево) достижимости. Это фундаментальный метод. Строится ориентированный граф,

вершинами которого являются все возможные маркировки, достижимые из начальной. Он наглядно показывает все возможные сценарии поведения системы.

- Матричные уравнения. Поведение сети можно описать с помощью матриц инцидентности. Срабатывание перехода соответствует прибавлению вектора, что позволяет решать задачи достижимости алгебраическими методами.
- Методы редукции. Сложная сеть упрощается путём замены стандартных подсетей (например, последовательных или параллельных фрагментов) на эквивалентные.

3.6 Обедающие философы

Задача «Обедающие философы» была сформулирована Эдсгером Дейкстрой в 1965 году как иллюстрация проблемы синхронизации в параллельных вычислительных системах. Она наглядно демонстрирует явления взаимной блокировки (deadlock) и голодания (starvation) при конкурентном доступе к разделяемым ресурсам. Это одна из самых известных задач, демонстрирующая проблемы параллелизма, в частности, взаимные блокировки (deadlocks) и состояние гонки (race conditions).

3.7 Классическая постановка

За круглым столом сидят N философов (обычно 5). Перед каждым философом стоит тарелка с едой. Между каждыми двумя соседними философами лежит одна вилка (или палочка для еды). Таким образом, количество вилок равно количеству философов (рис. 5.1). Правила поведения философов: - Философ может находиться в одном из трёх состояний: думает, голоден (хочет есть), ест. - Чтобы поесть, философу необходимы две вилки — та, что слева от него, и та, что справа. - Если философ не может получить обе вилки одновременно, он ждёт, пока они освободятся. - Поев, философ кладёт обе вилки обратно на стол и возвращается к размышлениям.

3.8 Классическая постановка

Ограничения:

- Вилками нельзя пользоваться одновременно двум философам.
- Философ не может отнять вилку у соседа — только дожидаться, пока тот её положит.

3.9 Проблема взаимной блокировки (deadlock)

При наивной реализации (каждый философ сначала берёт левую вилку, затем правую) возникает тупиковая ситуация: если все философы одновременно возьмут левую вилку, то каждый будет ждать правую, которая уже занята соседом. В результате никто не может поест — система замирает. Это и есть взаимная блокировка.

3.10 Решение проблемы

Для предотвращения deadlock предложены различные подходы: - Арбитр (официант) — вводится дополнительный процесс, который разрешает есть не более чем $N-1$ философам одновременно. - Асимметричное правило — философы с нечётным номером сначала берут левую вилку, затем правую, а чётные — наоборот. - Атомарный захват — философ берёт обе вилки одновременно (если они свободны) иначе не берёт ни одной. - Использование мьютексов и условных переменных в программных реализациях.

3.11 Связь с сетями Петри

Задача «Обедающие философы» является классическим примером для моделирования с помощью сетей Петри. Каждый философ и каждая вилка

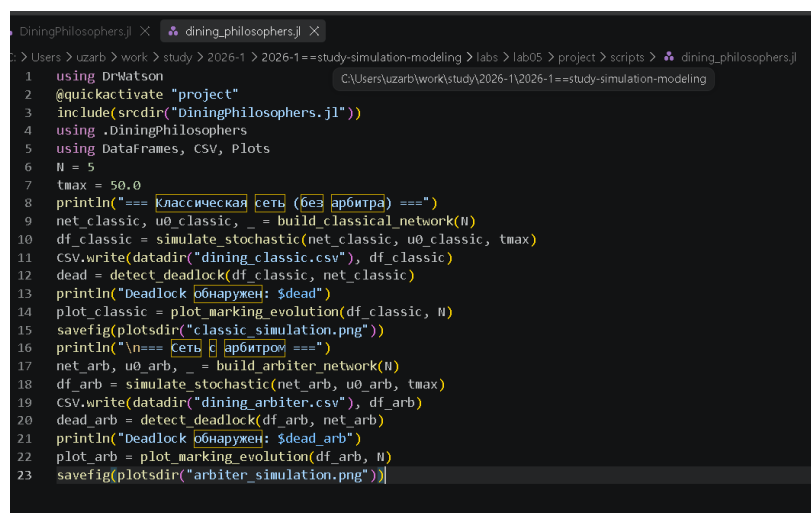
представляются позициями, а переходы соответствуют действиям: «взять левую вилку», «взять правую вилку», «начать есть», «положить вилки». Сеть Петри наглядно показывает возникновение deadlock и позволяет экспериментировать с различными механизмами синхронизации, например, введением арбитра.

3.12 Общие замечания

- Стохастичность: алгоритм Джиллесли вносит случайность. При каждом запуске могут получаться разные траектории и разное время наступления deadlock. Для надёжности рекомендуется проводить несколько прогонов (например, с разными seed) и усреднять результаты.
- Параметры N и $t_{\max}$: для $N=5$ deadlock в классической модели обычно наступает в течение 50 единиц времени.
- Если deadlock не обнаружился, можно увеличить $t_{\max}$.
- Сеть с арбитром.
- Теоретически она никогда не заходит в deadlock.
- Все три скрипта вместе дают полное исследование: от численного моделирования и визуализации динамики до сравнительного анализа и наглядной анимации.

4 Выполнение лабораторной работы

Создаем файл в project/src с кодом модели, который будем использоваться в дальнейших скриптах(рис. 4.1).



```
1 using DrWatson
2 @quickactivate "project"
3 include(scrdir("DiningPhilosophers.jl"))
4 using .DiningPhilosophers
5 using DataFrames, CSV, Plots
6 N = 5
7 tmax = 50.0
8 println("=== классическая сеть (без арбитра) ===")
9 net_classic, u0_classic, _ = build_classical_network(N)
10 df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
11 CSV.write(datadir("dining_classic.csv"), df_classic)
12 dead = detect_deadlock(df_classic, net_classic)
13 println("Deadlock обнаружен: $dead")
14 plot_classic = plot_marking_evolution(df_classic, N)
15 savefig(plotsdir("classic_simulation.png"))
16 println("\n=== сеть с арбитром ===")
17 net_arb, u0_arb, _ = build_arbiter_network(N)
18 df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
19 CSV.write(datadir("dining_arbiter.csv"), df_arb)
20 dead_arb = detect_deadlock(df_arb, net_arb)
21 println("Deadlock обнаружен: $dead_arb")
22 plot_arb = plot_marking_evolution(df_arb, N)
23 savefig(plotsdir("arbiter_simulation.png"))]
```

Рисунок 4.1: Код модели

4.1 Базовый эксперимент

Создаем файл с базовым экспериментом, который выполняет основное моделирование и сравнение двух вариантов сети Петри: - классическая модель (без арбитра), в которой возможна взаимная блокировка (deadlock); - модифицированная модель с арбитром, которая должна предотвращать deadlock.(рис. 4.2).

```

DiningPhilosophers.jl x dining_philosophers.jl
> Users > uzarb > work > study > 2026-1 ==study-simulation-modeling > labs > lab05 > project > scripts > dining_philosophers.jl
1 using DrWatson
2 @quickactivate "project"
3 include(sourcedir("DiningPhilosophers.jl"))
4 using .DiningPhilosophers
5 using DataFrames, CSV, Plots
6 N = 5
7 tmax = 50.0
8 println("=== Классическая сеть (без арбитра) ===")
9 net_classic, u0_classic, _ = build_classical_network(N)
10 df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
11 CSV.write(datadir("dining_classic.csv"), df_classic)
12 dead = detect_deadlock(df_classic, net_classic)
13 println("Deadlock обнаружен: $dead")
14 plot_classic = plot_marking_evolution(df_classic, N)
15 savefig(plotsdir("classic_simulation.png"))
16 println("\n=== Сеть с арбитром ===")
17 net_arb, u0_arb, _ = build_arbiter_network(N)
18 df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
19 CSV.write(datadir("dining_arbiter.csv"), df_arb)
20 dead_arb = detect_deadlock(df_arb, net_arb)
21 println("Deadlock обнаружен: $dead_arb")
22 plot_arb = plot_marking_evolution(df_arb, N)
23 savefig(plotsdir("arbiter_simulation.png"))

```

Рисунок 4.2: Базовая модель

4.2 Базовый эксперимент

Запускаем данный скрипт (рис. 4.3).

```

julia> exit()
PS C:\Users\uzarb\work\study\2026-1\study-simulation-modeling\labs\lab05\project\scripts> julia dining_philosophers.jl
Activating project at "C:\Users\uzarb\work\study\2026-1\study-simulation-modeling\labs\lab05\project"
=== Классическая сеть (без арбитра) ===
Deadlock обнаружен: true
|
| === Сеть с арбитром ===
Deadlock обнаружен: false
PS C:\Users\uzarb\work\study\2026-1\study-simulation-modeling\labs\lab05\project\scripts> |

```

Рисунок 4.3: Запуск

4.3 Базовый эксперимент

Далее создаем все производные форматы(рис. 4.4).

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts> julia tangle.jl dining_
_philosophers.jl
[ Info: generating project at 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project'
[ Info: generating plain script file from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\
project\scripts\dining_philosophers.jl'
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts
\dining_philosophers\dining_philosophers.jl'
[ Info: generating notebook from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts\dining_p
hilosophers\dining_philosophers.jl'
[ Info: generating markdown page from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\pro
ject\scripts\dining_philosophers.jl'
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\markdow
n\dining_philosophers\dining_philosophers.qmd'
[ Info: generating notebook from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\
scripts\dining_philosophers.jl'
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\noteboo
ks\dining_philosophers\dining_philosophers.ipynb'
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\notebooks\dining_phil
osophers\dining_philosophers.ipynb'
Готово! Все файлы созданы.
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts>
```

Рисунок 4.4: Код модели

Запускаем файл notebook(рис. 4.5).

```
Jupyter dining_philosophers Last Checkpoint 53 seconds ago

File Edit View Run Kernel Settings Help

JupyterLab Julia 1.12

[1]: using Orthon
@docactivate "project"
include(srcdir("DiningPhilosophers.jl"))
using DiningPhilosophers
using DataFrames, CSV, Plots
N = 5
tmax = 50.0
println("=== Классическая сеть (без арбитра) ===")
net_classic, u0_classic, _ = build_classical_network(N)
df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
CSV.write(srcdir("dining_classic.csv"), df_classic)
dead = detect_deadlock(df_classic, net_classic)
println("Deadlock обнаружен: $dead")
plot_classic = plot_marking_evolution(df_classic, N)
savefig(plotdir("classic_simulation.png"))
println("==== Сеть с арбитром ====")
net_arb, u0_arb, _ = build_arbiter_network(N)
df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
CSV.write(srcdir("dining_arbiter.csv"), df_arb)
dead_arb = detect_deadlock(df_arb, net_arb)
println("Deadlock обнаружен: $dead_arb")
plot_arb = plot_marking_evolution(df_arb, N)
savefig(plotdir("arbiter_simulation.png"))
=== Классическая сеть (без арбитра) ===
```

Рисунок 4.5: notebook

В результате получаем следующие графики(рис. 4.6).(рис. 4.7).

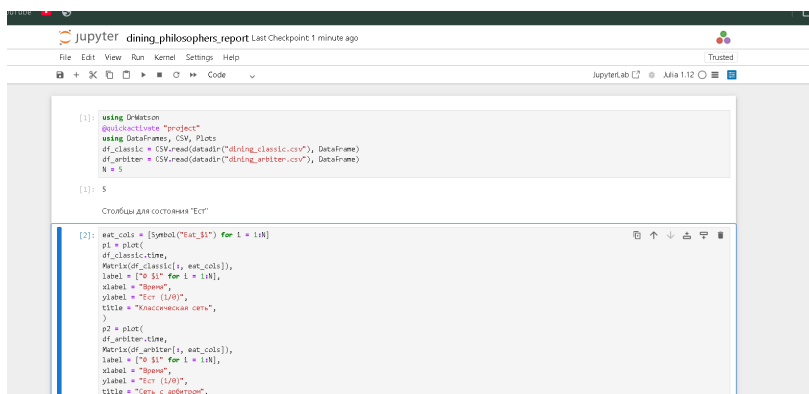


Рисунок 4.6: График классической модели

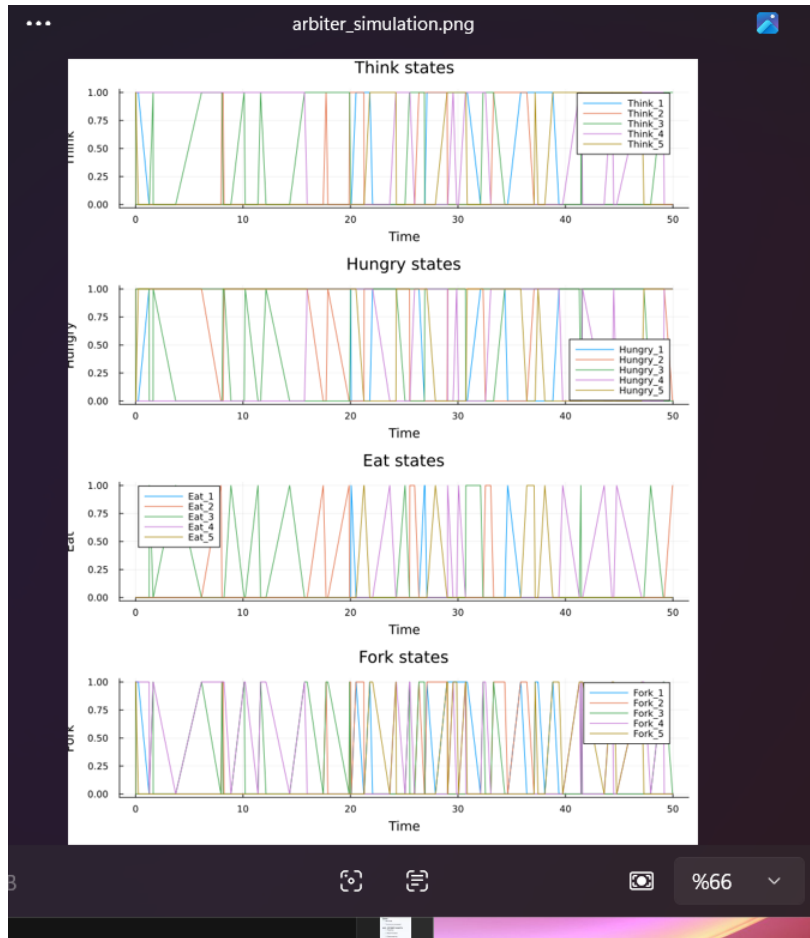


Рисунок 4.7: График модели с арбитром

4.4 Анимация процесса

Создаем следующий скрипт, который сделает gif файл с анимацией процесса и наглядно продемонстрирует динамику работы сети петри(рис. 4.8).

```

=== Сеть с арбитром ===
Deadlock обнаружен: false
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project> julia dining_philosophers_animation
.jl
[ Info: Saved animation to C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\plots\philosophers_s
imulation.gif
* Анимация сохранена в plots\philosophers_simulation.gif
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project>

```

Рисунок 4.8: Код анимации

4.5 Анимация процесса

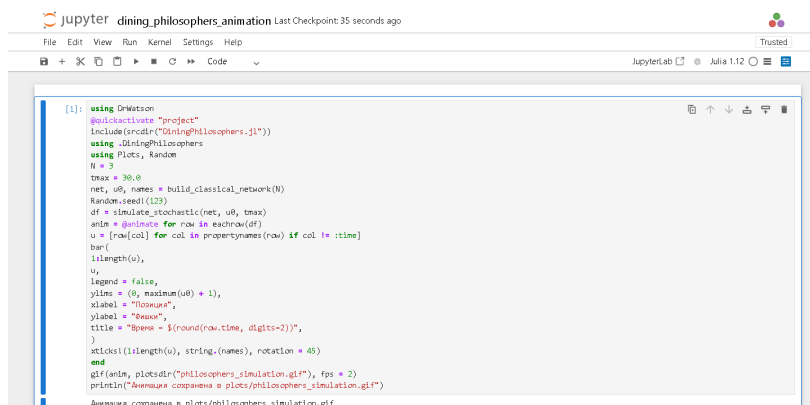
Запускаем и создаем все производные форматы(рис. 4.9).

```
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project> julia tangle.jl dining_philosophers_animation.jl
Activating project at 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project'
Генерация из: dining_philosophers_animation.jl
[ Info: generating plain script file from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts\dining_philosophers_animation.jl'
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts\dining_philosophers_animation\dining_philosophers_animation.jl'
✓ Includer example: C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts\dining_philosophers_animation\dining_philosophers_animation.jl
[ Info: generating markdown page from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts\dining_philosophers_animation.jl'
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\markdown\dining_philosophers_animation\dining_philosophers_animation.qmd'
✓ Quarto: C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\markdown\dining_philosophers_animation\dining_philosophers_animation.qmd
[ Info: generating notebook from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts\dining_philosophers_animation.jl'
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\notebooks\dining_philosophers_animation\dining_philosophers_animation.ipynb'
✓ Notebook: C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\notebooks\dining_philosophers_animation\dining_philosophers_animation.ipynb
Готово! Все файлы созданы.
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts>
```

Рисунок 4.9: производные форматы

4.6 Анимация процесса

Запускаем файл notebook(рис. 4.10).



```
[1]: using Orthon
@docactivate "project"
include(scrdir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using Plots, Random
N = 3
tmax = 30.0
net, u0, names = build_classical_network(N)
Random.seed!(123)
df = simulate_stochastic(net, u0, tmax)
anim = @animate for row in eachrow(df)
u = [row[col] for col in propertynames(row) if col != :time]
bar(
    llength(u),
    u,
    legend = false,
    ylims = (0, maximum(u0) + 1),
    xlabel = "Позиция",
    ylabel = "Голова",
    title = "Время = $(round(row.time, digits=2))",
)
xticks(llength(u), string.(names), rotation = 45)
end
gif(anim, plotsdir("philosophers_simulation.gif"), fps = 2)
println("Анимация сохранена в plots/philosophers_simulation.gif")
Анимация сохранена в plots/philosophers_simulation.gif
```

Рисунок 4.10: Код модели

4.7 Итоговый отчет

Создаем и запускаем скрипт для итогового отчета и сравнительного анализа двух моделей(рис. 4.11).

```
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts> julia dining_philosophers_report.jl
Activating project at 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project'
Отчет сохранен в plots\final_report.png
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts>
```

Рисунок 4.11: Запуск

4.8 Итоговый отчет

Создаем все производные форматы(рис. 4.10).

```
Отчет сохранен в plots\final_report.png
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts> julia tangle.jl dining_philosophers_report.jl
Activating project at 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project'
Генерация из: dining_philosophers_report.jl
[ Info: generating plain script file from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts\dining_philosophers_report.jl'
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts\dining_philosophers_report\dining_philosophers_report.jl'
[ Info: generating markdown page from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts\dining_philosophers_report\dining_philosophers_report.jl'
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\markdown\dining_philosophers_report\dining_philosophers_report.qmd'
[ Info: generating notebook from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts\dining_philosophers_report\dining_philosophers_report.jl'
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\notebooks\dining_philosophers_report\dining_philosophers_report.ipynb'
[ Info: Notebook: C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\notebooks\dining_philosophers_report\dining_philosophers_report.ipynb
Готово! Все файлы созданы.
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab05\project\scripts>
```

Рисунок 4.12: производные форматы

4.9 Итоговый отчет

Запускаем файл notebook(рис. 4.13).

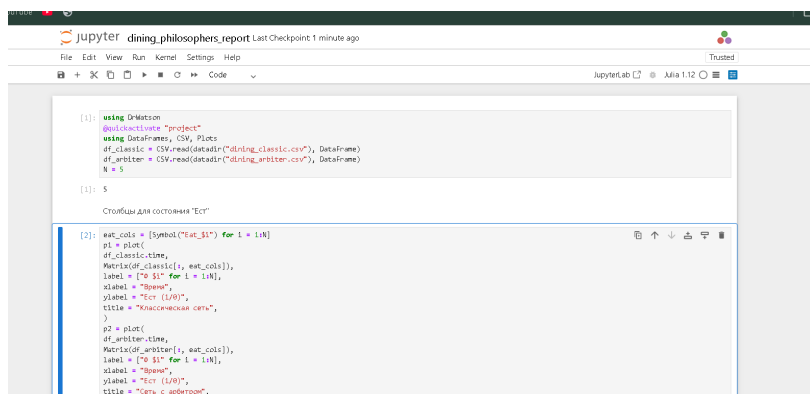


Рисунок 4.13: notebook

В результате получаем график (рис. 4.14).

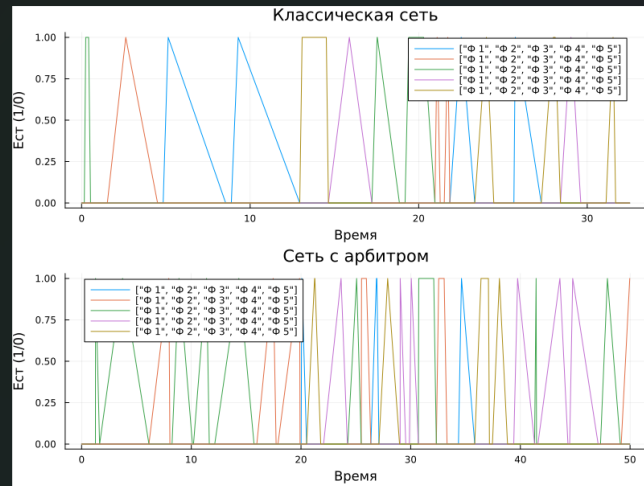


Рисунок 4.14: График

5 Выводы

После выполнения данной лабораторной работы мы познакомились с математическим аппаратом Сеть Петри и задачей об обедающих философах

Список литературы